

College of Computing

Computer Science Department

Project Event Management CSC382

- Student Information

Student 1:	**Salem Hamdan Salem Al-Omrani**	**ID:**	**202322090**
Student 2:	**Abdullah Ali Khalaf Al-Rashidi**	**ID:**	**202422065**
Student 3:	**Ayman Abdullah Al-Omari**	**ID:**	**202412014**

Software Requirements Specification (SRS) for the Event Management System

1. Introduction

This Software Requirements Specification (SRS) document details the functional and non-functional requirements for the Event Management System (EMS), a web-based application designed to streamline the process of event creation, management, and attendee registration [1]. The system is developed using a standard open-source technology stack, specifically PHP and MySQL, and is intended for local deployment, as indicated by the project's setup instructions [Project README]. This specification adheres to the principles outlined in the IEEE 830 standard for defining clear, unambiguous, and verifiable software requirements, ensuring alignment between stakeholder expectations and the final product [2].

2. Stakeholder Analysis

A comprehensive analysis of the system's stakeholders is crucial for defining the scope and prioritizing requirements, as their roles dictate the necessary access controls and system functionalities [3]. The EMS

primarily serves three distinct user groups, each with specific needs and permissions that govern their interaction with the system.

Stakeholder Group	Role and Description	Key Responsibilities and Permissions
Guest/Public User	Any individual accessing the system without logging in. This group represents potential event attendees.	**Read-Only Access:** View the main event list (index.php) and view detailed information for any specific event (event_view.php) [Project README].
Registered User/Attendee	An individual who has an account and logs into the system.	**Registration:** Register their attendance for an event (register.php). **Authentication:** Log in (login.php) and log out (logout.php) [4].
Administrator/Organizer	Users with elevated privileges responsible for the content and management of the system.	**Full CRUD Operations:** Create, Edit, and Delete events (event_create.php, event_edit.php). **Management:** Access the administrative dashboard (admin.php) to view and manage registrations [Project README].

3. System Scope and Constraints

3.1. System Scope

The primary scope of the EMS is to provide a centralized, web-based platform for managing the core lifecycle of events, from creation to attendee registration [5]. The system's boundaries are defined by its core functionalities: event listing, detailed event viewing, user authentication, event creation/modification by authorized personnel, and simple attendee registration. The system is designed to be a self-contained unit, with no immediate requirement for integration with external payment gateways or advanced analytics platforms, although its modular structure allows for future expansion [6].

3.2. System Constraints

The development and deployment of the EMS are subject to several technical and operational constraints, which must be considered during implementation and maintenance [7].

- **Technology Stack Constraint:** The system is constrained to the use of PHP (version >= 7.4) and a local MySQL database, which dictates the programming environment and database management capabilities [Project README].
- **Deployment Constraint:** The system is designed for local deployment, utilizing a local web server (e.g., PHP built-in server or Apache/Nginx), which limits its initial scalability and public accessibility [Project README].
- **Security Constraint:** All user passwords must be stored using the password_hash function, and all database interactions must utilize Prepared Statements to prevent SQL injection attacks [8].
- **File Upload Constraint:** Image uploads are restricted to a maximum size of 2MB and specific formats (JPG, JPEG, PNG, WEBP), as defined in the project's image management specifications [Project README].
-

4. Functional Requirements (FR)

Functional requirements specify the functions that the system must perform to satisfy the user's needs [9]. These requirements are derived directly from the system's intended use and the identified stakeholder roles.

ID	Requirement Description	Actor(s)	Source File/Module
FR-1.0	**Authentication Management**		
FR-1.1	The system shall allow a user to log in using a valid username and password.	Registered User, Administrator	login.php, auth.php
FR-1.2	The system shall terminate the user's session upon request (logout).	Registered User, Administrator	logout.php
FR-2.0	**Event Viewing and Search**		
FR-2.1	The system shall display a list of all available events on the homepage.	Guest, Registered User	index.php

FR-2.2	The system shall allow users to view the full details of a selected event.	Guest, Registered User	pages/event_view.php
FR-2.3	The system shall provide functionality to search and sort the event list by date or title.	Guest, Registered User	index.php
FR-3.0	**Event Creation and Modification**		
FR-3.1	The system shall allow Administrators/Organizers to create a new event, including title, description, date, and image upload.	Administrator	pages/event_create.php
FR-3.2	The system shall allow Administrators/Organizers to modify the details of an existing event.	Administrator	pages/event_edit.php
FR-3.3	The system shall allow Administrators/Organizers to delete an existing event.	Administrator	pages/event_edit.php
FR-4.0	**Registration Management**		
FR-4.1	The system shall allow a Registered User to register their attendance for a specific event.	Registered User	pages/register.php
FR-4.2	The system shall allow the Administrator to view and manage the list of registered attendees for all events.	Administrator	pages/admin.php

5. Non-Functional Requirements (NFR)

Non-functional requirements define the quality attributes of the system, specifying how well the system performs its functions [10]. These are critical for user satisfaction and system longevity.

5.1. Security (NFR-S)

Security requirements are paramount, especially given the handling of user authentication data [11].

- **NFR-S.1 (Authentication):** The system shall store all user passwords using a strong, one-way hashing algorithm (password_hash) to prevent plaintext exposure [Project README].
- **NFR-S.2 (Input Validation):** The system shall implement server-side validation and utilize Prepared Statements for all database queries to mitigate SQL injection attacks [8].
- **NFR-S.3 (Cross-Site Scripting - XSS):** The system shall use htmlspecialchars on all user-supplied output to prevent Cross-Site Scripting (XSS) vulnerabilities [Project README].
- **NFR-S.4 (Cross-Site Request Forgery - CSRF):** The system shall implement CSRF tokens on all state-changing forms (e.g., event creation, registration) to prevent unauthorized command execution [12].

5.2. Performance (NFR-P)

Performance requirements ensure the system responds quickly and efficiently under expected load [13].

- **NFR-P.1 (Response Time):** The system shall load the main event list (index.php) within 3 seconds under a normal load of 50 concurrent users.
- **NFR-P.2 (Query Efficiency):** All database queries, particularly those for event listing and registration lookup, shall be optimized to execute in less than 500 milliseconds.

5.3. Reliability (NFR-R)

Reliability requirements define the system's ability to operate without failure over a specified period [14].

- **NFR-R.1 (Availability):** The system shall maintain an uptime of 99.9% during scheduled operational hours, excluding planned maintenance windows.
- **NFR-R.2 (Data Integrity):** The system shall ensure transactional integrity for event creation and registration, preventing partial data commits in the event of a system failure.

5.4. Usability (NFR-U)

Usability requirements focus on the ease with which users can interact with the system [15].

- **NFR-U.1 (User Interface):** The system shall utilize a clean, responsive design (CSS/JavaScript) to ensure optimal viewing and interaction across various devices [Project README].
- **NFR-U.2 (Accessibility):** The system shall provide clear and informative error messages for all failed operations (e.g., login failure, invalid form submission).

6. Use Cases

Use cases describe the interaction between an actor and the system to achieve a specific goal [16]. The following table outlines the primary use cases for the EMS.

Use Case ID	Use Case Name	Actor(s)	Description
UC-1	**View Event List**	Guest, Registered User	The actor accesses the homepage to browse all available events. The system displays a paginated list with basic details (title, date).
UC-2	**Register for Event**	Registered User	The actor logs in, selects an event, and submits the registration form. The system records the attendance in the database and provides a confirmation [FR-4.1].
UC-3	**Create New Event**	Administrator	The administrator logs in, navigates to the creation page, enters all required event details (including image upload), and submits the form. The system validates the input and adds the new event to the database [FR-3.1].
UC-4	**Manage Registrations**	Administrator	The administrator logs in and accesses the admin dashboard. The system displays a list of all events and their corresponding registered attendees, allowing for management actions [FR-4.2].

UC-5	**Log In to System**	Registered User, Administrator	The actor navigates to the login page and provides valid credentials. The system authenticates the user and redirects them to the appropriate dashboard or homepage [FR-1.1].

References

[Project README]: Project README. (n.d.). *Event Management CSC382 README*. [Local Project Documentation].

[1]: IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*.

[2]: IEEE. (1993). *IEEE Std 830-1993*. [https://www.utdallas.edu/~chung/RE/IEEE830-1993.pdf].

[3]: GeeksforGeeks. (2024). *What is Stakeholder? Definition, Types and Examples*. [https://www.geeksforgeeks.org/software-engineering/stakeholder-software-engineering/]. [4]: Concepta Tech. (2023). *How to Define Stakeholders for Your Software Development Project*. [https://www.conceptatech.com/blog/how-to-define-stakeholders-for-your-software-development-project]. [5]: QAT. (2025). *Managing Stakeholder-Driven Changes in Software Requirements*. [https://qat.com/managing-stakeholder-driven-changes-srs/].

[6]: LinkedIn. (2023). *What role do stakeholders play in requirements engineering and software development?*. [https://www.linkedin.com/advice/3/what-role-do-stakeholders-play-requirements-engineering-1tsee].

[7]: Rebus Community. (n.d.). *Appendix C: IEEE 830 Template*. [https://press.rebus.community/requirementsengineering/back-matter/appendix-c-ieee-830-template/].

[8]: OWASP Foundation. (n.d.). *SQL Injection Prevention Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html]. [9]: GeeksforGeeks. (2025). *Functional and Non Functional Requirements*. [https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/].

[10]: Requiment. (2024). *What are Functional and Non-functional Requirements?*. [https://www.requiment.com/what-are-functional-and-non-functional-requirements/].

[11]: OWASP Foundation. (n.d.). *OWASP Top Ten*. [https://owasp.org/www-project-top-ten/].

[12]: OWASP Foundation. (n.d.). *Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html].

[13]: Jama Software. (n.d.). *Functional vs. Non-Functional Requirements*. [https://www.jamasoftware.com/requirements-management-guide/writing-requirements/functional-vs-non-functional-requirements/].

[14]: Medium. (n.d.). *Functional and Non-Functional Requirements: The Ultimate Checklist with Examples*. [https://medium.com/@growsolutions/functional-and-non-functional-requirements-the-ultimate-checklist-with-examples-cde16aba33d7].

[15]: ISO. (2018). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.

[16]: OnlineExpo. (n.d.). *20 must-have functions for event management software*. [https://onlineexpo.com/en/articles/20-must-have-functions-for-event-management-software/].